

Scientific computing in high-energy physics

Lecture 1, 16.04.2026

Ante Bilandzic
(E62, Dense and Strange Hadronic Matter)



Outline of today's lecture

- Course trivia
- Free adverts
 - Linux
 - Bash
 - ROOT



Course trivia

- **PH8124:** ‘Scientific computing in high-energy physics’
 - <https://academics.nat.tum.de/org/mh/details/mod/PH8124/>
- When & where:
 - Thursday: 14:00-16:00 (hybrid mode)
 - Physics Department, E62 seminar room 2024
 - Roomfinder: <https://nav.tum.de/room/5101.EG.024>
 - 12+ contact days (the last lecture is on July 16th)
- Examination:
 - Homeworks (10 in total)
 - Oral examination during the presentation of the final project
- Contact:
 - Ante Bilandzic, ante.bilandzic@tum.de
 - Office PH 2101: <https://nav.tum.de/room/5101.EG.101>

Course trivia

- The presence in person is encouraged
- Parallel online coverage via **Microsoft Teams (MT)**
- Why not **Zoom**?
 - As of SS2026, TUM no longer offers the licensed Zoom version...
- Recurring **MT** meetings on Thursdays, from 13:30-16:00
 - **MT** coordinates will be distributed via the official mailing list to all registered students for this course
- Per popular request, lectures are recorded, and recordings shared immediately afterward via email



Microsoft Teams

Webpage

- The course has a dedicated webpage:
 - <https://abilandz.gitbook.io/ss2026> (GitBook version)

 **SS2026**

Ctrl

K

README

Lecture 1: Trivia

Lecture 2: Commands and variables

Lecture 3: Linux file system. Positional parameters. Your first Linux/Bash command. Command precedence

Lecture 4: Loops and few other things

Lecture 5: Command substitution. Input/Output (I/O). Conditional statements

Lecture 6: String manipulation. Arrays. Pipes. sed, awk and grep

Lecture 7: Escaping. Quotes. Handling processes and jobs

Lecture 8: Bash fancy features

Lecture 9: Real-life examples

Lecture 10: ROOT - getting started

Lecture 11: ROOT - basic classes (Part 1/2)

Lecture 12: ROOT - basic classes (Part 2/2)

Homeworks

Final Project

README

Copy

Last update: 20260323

This webpage contains the lecturing material for the course PH8124, 'Scientific computing in high-energy physics', offered in SS2026.

The material covered in the previous year is at [SS2025](#) (the current semester is based on it, only minor modifications are foreseen).

The formal course description can be found at the TUM website at the following internal [link](#), and at the following public [link](#).

For **ROOT**, the official documentation is used:

- Overview of all tutorials: <https://root.cern/manual/>
- Primer (for beginners): <https://root.cern/primer/> (or [pdf](#) version)
- Users Guide (last update 2018, not maintained anymore): [html](#) or [pdf](#) version

Next

Lecture 1: Trivia

>

Last updated 1 day ago

 Powered by GitBook

Webpage

- The course has a dedicated webpage:
 - <https://abilandz.github.io/PH8124/> (GitHub Pages version)

PH8124

Introduction

Last update: 20260323

This webpage contains the lecturing material for the course PH8124, 'Scientific computing in high-energy physics', offered in SS2026.

The material covered in the previous year is at [SS2025](#) (the current semester is based on it, only minor modifications are foreseen).

The formal course description can be found at the TUM website at the following internal [link](#), and at the following public [link](#).

For **ROOT**, the official documentation is used:

- Overview of all tutorials: <https://root.cern/manual/>
- Primer (for beginners): <https://root.cern/primer/> (or [pdf](#) version)
- Users Guide (last update 2018, not maintained anymore): [html](#) or [pdf](#) version

Lectures, homeworks and final project

- [Lecture 1: Trivia](#)
- [Lecture 2: Commands and variables](#)
- [Lecture 3: Linux file system. Positional parameters. Your first Linux/Bash command. Command precedence](#)
- [Lecture 4: Loops and few other things](#)
- [Lecture 5: Command substitution. Input/Output \(I/O\). Conditional statements](#)
- [Lecture 6: String manipulation. Arrays. Pipes. sed, awk and grep](#)
- [Lecture 7: Escaping. Quotes. Handling processes and jobs](#)
- [Lecture 8: Bash fancy features](#)
- [Lecture 9: Real-life examples](#)
- [Lecture 10: ROOT - getting started](#)
- [Lecture 11: ROOT - basic classes \(Part 1/2\)](#)
- [Lecture 12: ROOT - basic classes \(Part 2/2\)](#)

Course trivia

- After each lecture, an executive summary of the covered material will be shared via email
- The course webpage will be updated regularly
- In the same way, I will also share the homework exercises
- Recommended literature:
 - Mendel Cooper: *'Advanced Bash-Scripting Guide'*
(<http://tldp.org/LDP/abs/abs-guide.pdf>)
 - Cameron Newham and Bill Rosenblatt, *'Learning the bash Shell: Unix Shell Programming (In a Nutshell (O'Reilly))'*
 - Richard Blum and Christine Bresnahan, *'Linux Command Line and Shell Scripting Bible'*
 - ROOT tutorials (<https://root.cern/manual/>)

Course trivia

- Grading:
 - 3 ECTS points
 - Final grade = grade at the final project examination + 'one intermediate stepping of 0.3 to the better grade' if you have completed correctly 75% of all homeworks
 - There will be in total 10 homework exercises, one after each lecture
- Oral examination at the final project presentation:
 - The topic for the final programming project will be offered at some point towards the end of the lecture
 - The oral exam of about 25 minutes consists of presenting:
 1. How your programme was designed/implemented?
 2. Testing the execution of your code (crash-free, bug-free, efficiency in terms of CPU usage and memory consumption)
 3. Testing the code flexibility (e.g. how you would add some new feature in the code?)

Course trivia

- Preliminary list of topics to be covered:
 - **Linux:** filesystem hierarchy and file manipulation, handling processes and jobs, frequently used commands, etc.
 - **Bash:** shell environment, variables, string manipulation, built-in commands, aliases, functions, conditional statements, loops, command substitution, command chain, test constructs, piping, redirections, code blocks, subshells, process substitution, brace expansion, regular expressions, here-strings and here-documents, etc.
 - **ROOT:** using ROOT GUI, plotting, histogramming, functions, fitting, trees, file merging, writing executables, etc.

Course trivia

- Course classification:
 - At the moment, classified as a 'Non-physics elective course'
 - Open both to Bachelor and Masters students
 - <https://academics.nat.tum.de/en/msc/ph/nonphys>
- Course evaluation:
 - At some point during the lecture, you will be asked to evaluate this course: Please, do it! (reminder will be sent later)

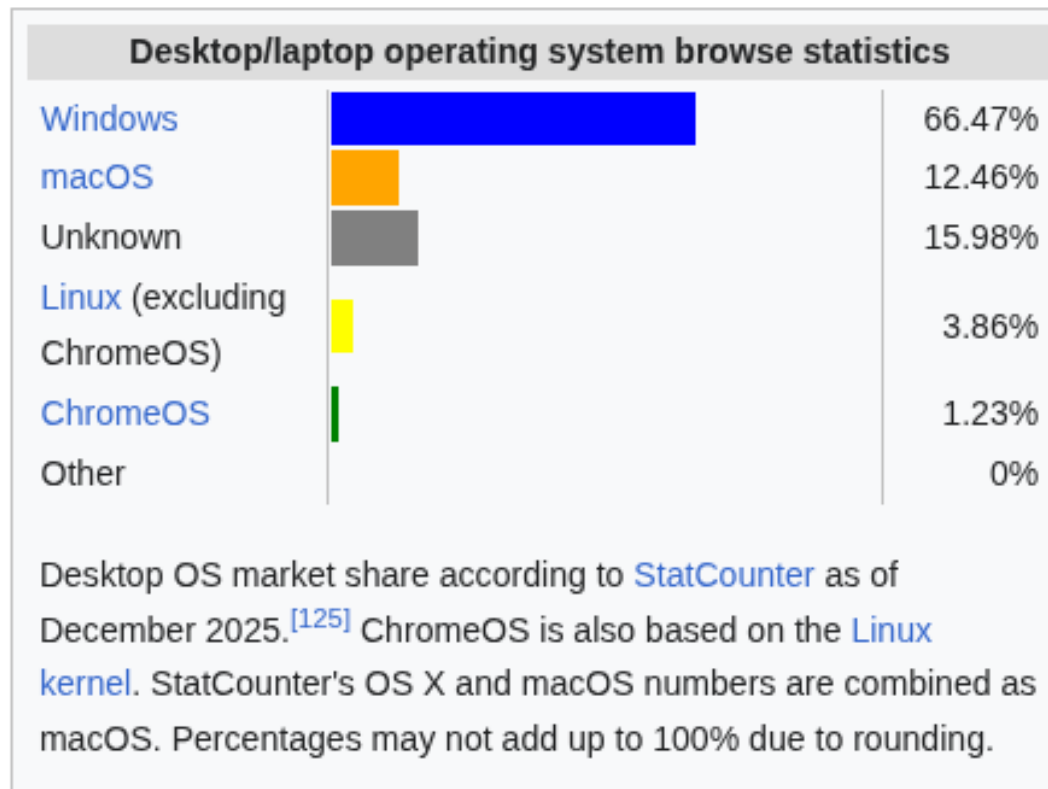
Trivia

- No lecture on:
 - May 14th – Ascension Day
 - June 04th – Corpus Christi

Free adverts

Free advert (#1)

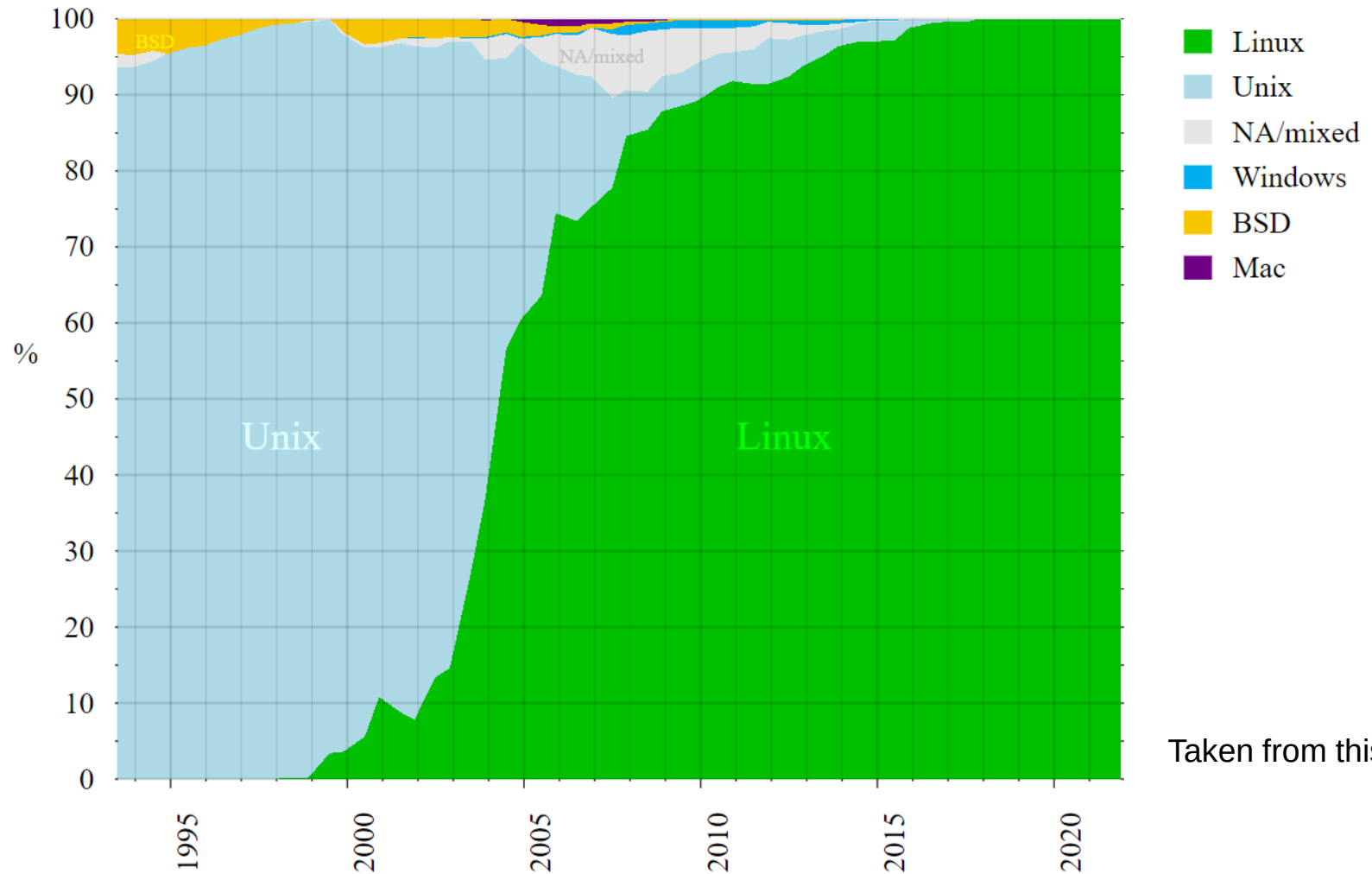
- Why Linux? Statistics for all desktop/laptop computers:



Taken from this [link](#)

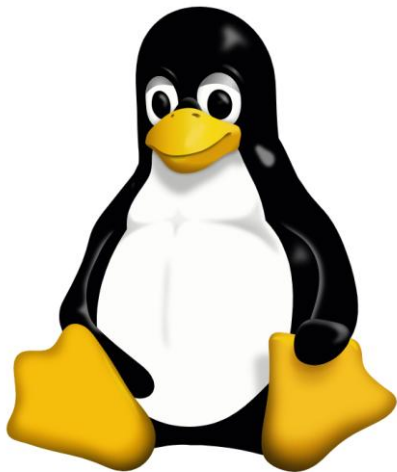
Free advert (#1)

- Why Linux? Statistics for supercomputers:



Free advert (#1)

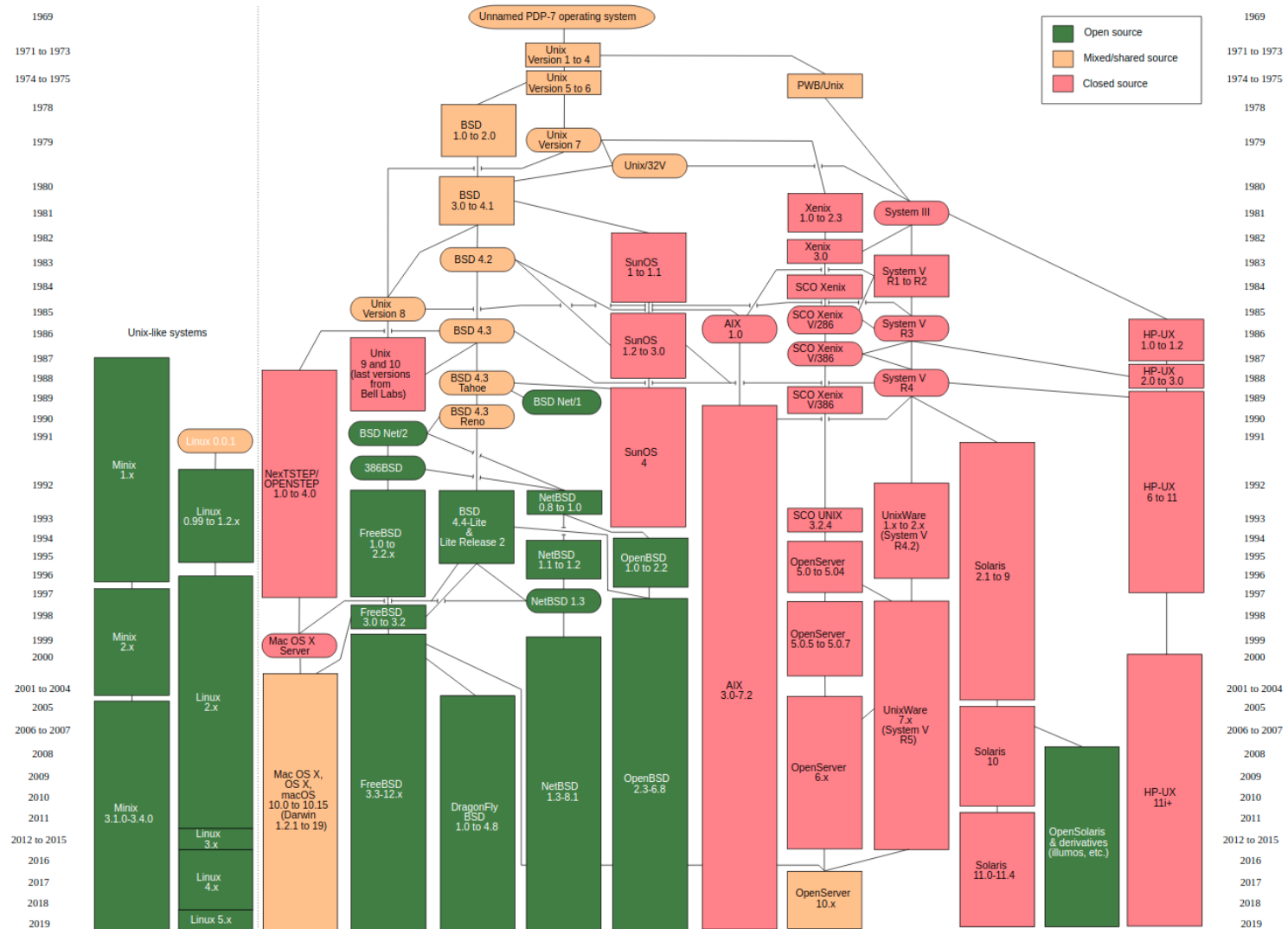
- Why Linux?
- Linux is by far the leading operating system in computers used in scientific research (CERN, NASA, etc.)
- **Linux kernel:** developed by Linus Torvalds in the early 90s
- **GNU ('GNU's not Unix'):** open source utilities to perform standard actions on the computer – no licensing
 - Linux kernel + GNU utilities = Linux operating system
- Initially, the Linux OS was basically a free Unix



Penguin named Tux is the most commonly used logo for Linux

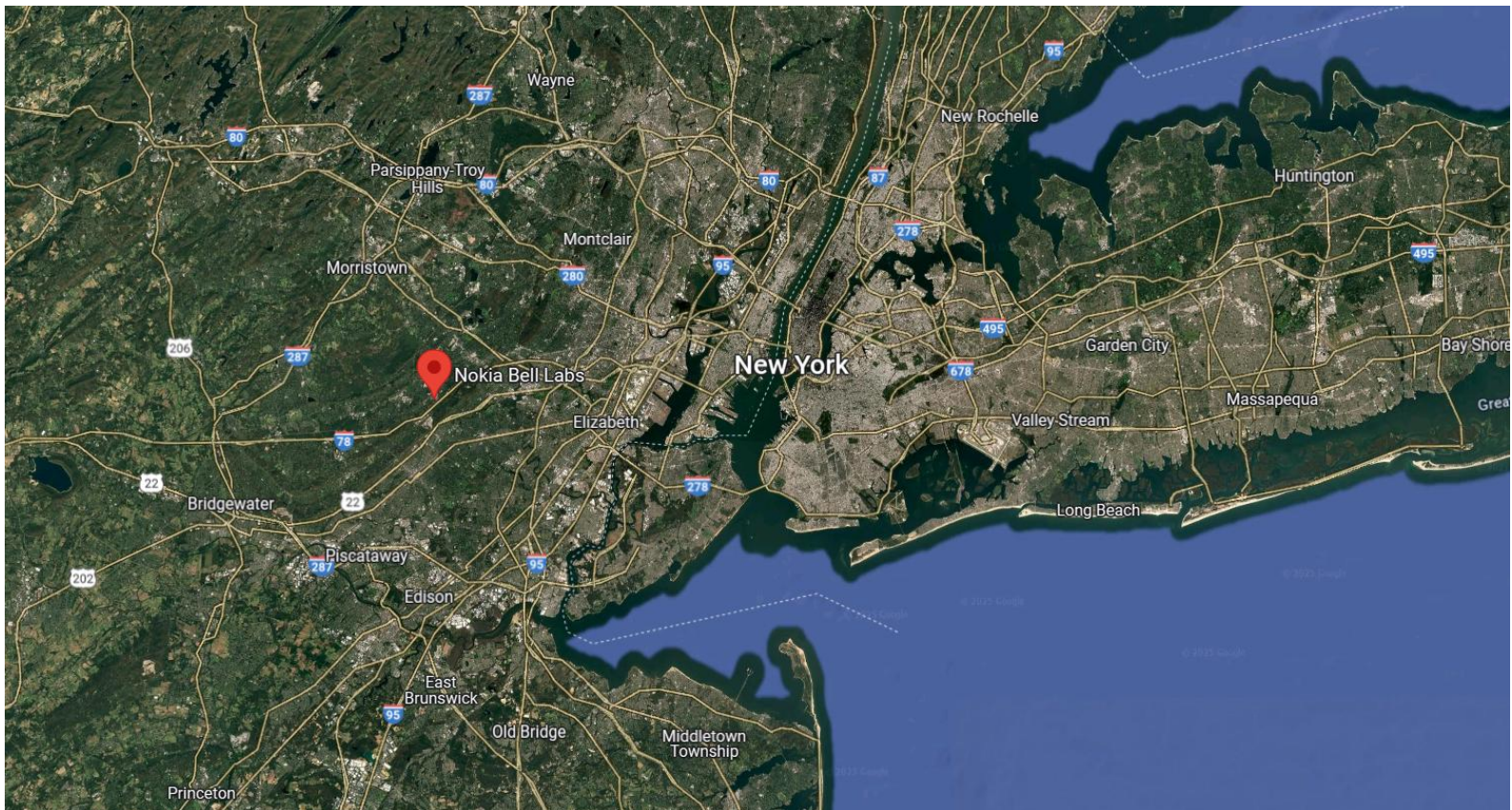
Free advert (#1)

- Unix family tree – Linux is one of its descendants



Free advert (#1)

- Unix was developed at “Bell Labs” in Murray Hill, New Jersey, USA, in the 70’s



Free advert (#1)

- Unix was developed at “Bell Labs” in Murray Hill, New Jersey, USA, in the 70’s



Free advert (#1)

- Linux family tree: Common Linux kernel and plethora of different Linux distributions built on top of it
 - Full-core: Debian, Red Hat Enterprise, openSUSE, etc.
 - Specific: Ubuntu, Fedora, CentOS, Linux Mint, etc.
- Specific distributions are derivatives of full-core distributions:
 - Ubuntu is derivative of Debian, Fedora of Red Hat, etc. (see complete tree at <https://distrowatch.com/images/other/distro-family-tree.png>)
- The material presented in this course will be demonstrated on Ubuntu, but it applies also to any other Linux distribution
- Regular updates on all distributions: <https://distrowatch.com>



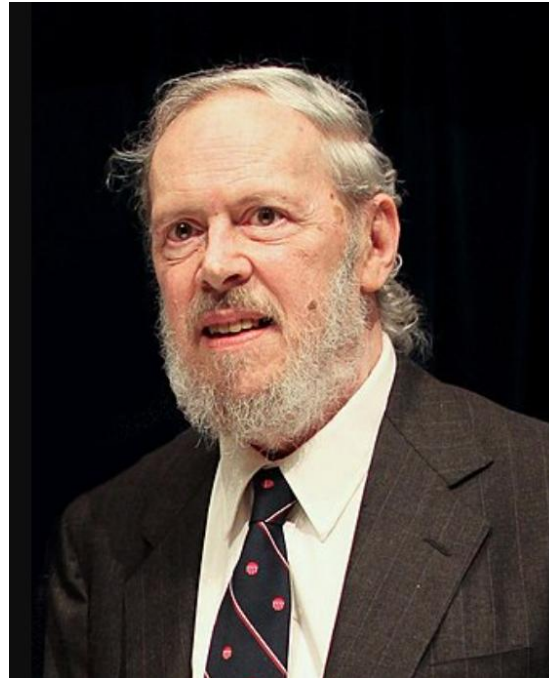
Key people

- Ken Thompson



B programming language,
UNIX operating system,
UTF-8 encoding,
Go programming language,
Chess tablebases,
...

- Denis Ritchie



C programming language,
UNIX operating system,
...

- Linus Torvalds



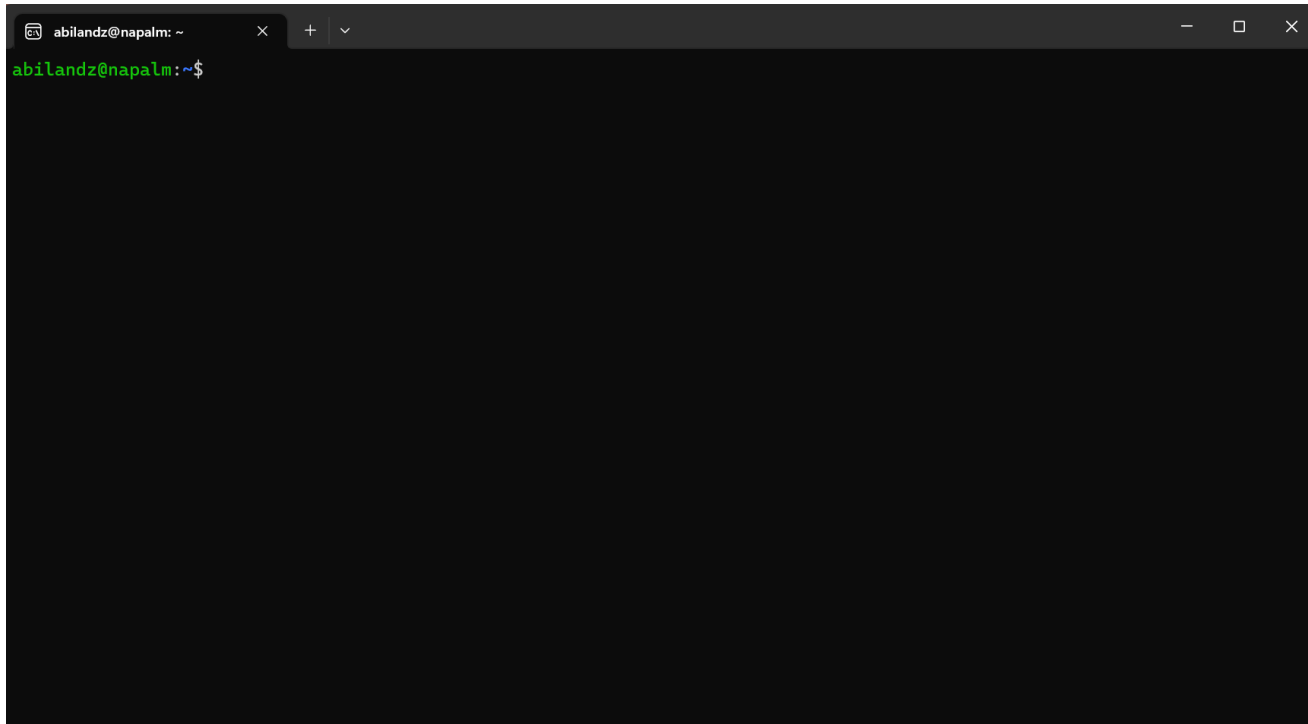
Linux operating system,
Git (version control system),
...

Getting Ubuntu for Windows users

- If you have only Windows on your laptop, you could either:
 - on Windows 11, use [Windows Subsystem for Linux](#) :
 - Open Windows PowerShell and list available Linux distributions with the command:
wsl --list --online
 - Pick up your favorite Linux distribution, and install it e.g. as follows:
wsl --install Ubuntu-24.04
 - The rest is easy (pick up your account, etc.)
 - or, install PuTTY (<https://www.putty.org/>) and then use it to connect and work remotely on some computer running Linux, on which you have an account

Free advert (#2)

- Is this the right course for you?
 - If, after you have opened a terminal in Linux ...



... you have asked yourself: 'What now?', then this is the right course for you!

Free advert (#2)

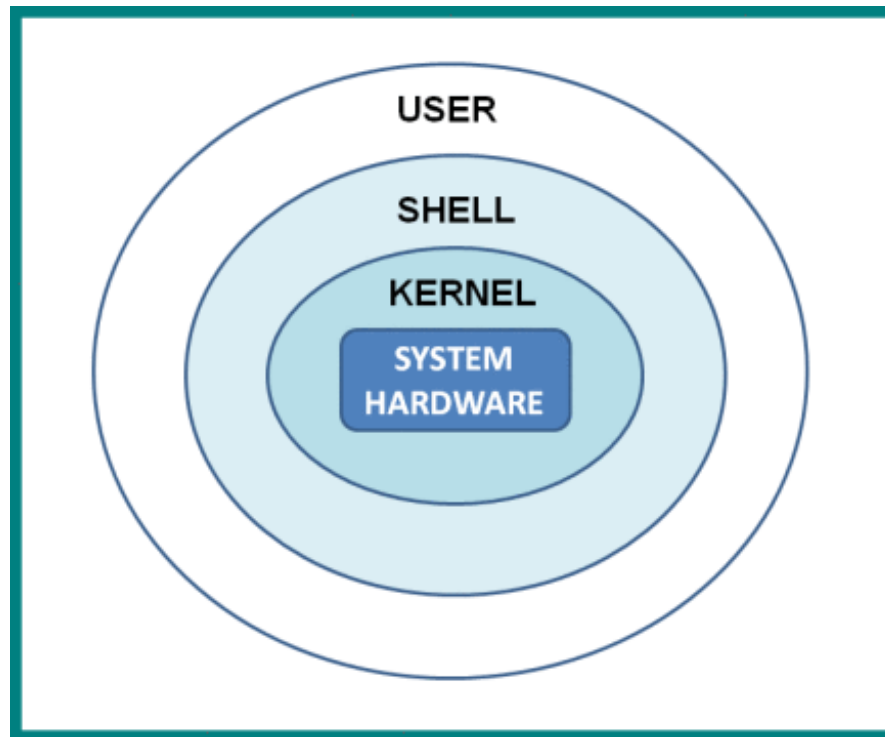
- What can we do in the terminal?
 - Not that much with the mouse...
- Next, you can start typing and pressing 'Enter', but most likely, whatever you have typed in the terminal will produce only error messages
 - Still, that is something, as it clearly means that there is some secret/magic language that is trying to respond to, or to interpret, your command input, as soon as you have typed something in the terminal and pressed 'Enter'. What is that secret built-in language available in the terminal?

Linux shells

- Loosely speaking, a shell is any program that a user employs to type commands in the terminal (text window)
- Example shells:
 - sh
 - bash
 - zsh
 - ksh
 - csh
 - fish
 - PowerShell (developed by Microsoft)
- Since Bash is the default shell on most Linux distributions nowadays, we focus on it
 - If not set by default, just type **bash** in the terminal, and you are in the Bash wonderland

Why shell?

- The shell translates the commands you type into a format that the computer can understand
 - It works both ways: The user is shielded from the Linux kernel, and the Linux kernel is shielded from the user



A bit of Bash history

- Initial development by Brian Fox in 1989 as a part of GNU project... And it's still alive!
- Bash is an acronym for 'Bourne-again shell'
 - The original shell 'sh' was written in 1977 by Stephen Bourne
- Written entirely in C
 - The Linux kernel is mostly written in C
- Executable: /bin/bash
- File extension: .sh
- Command processor / interpreted / scripting language



The current status of Bash

- Bash is well-maintained and is under regular development
 - Webpage: <https://www.gnu.org/software/bash/>
 - Source code: <http://git.savannah.gnu.org/cgit/bash.git>
- Latest release: version 5.3 (July 3rd, 2025)
 - The current maintainer: Chet Ramey



Interpreted vs. compiled languages

- Interpreted:
 - write code & execute line-by-line
 - less reliable (there is no compiler to catch the errors!)
 - the source code can be easily read and copied
 - examples: Bash, Python, Mathematica, JavaScript
- Compiled:
 - write code & compile & execute the compiled file ('binaries')
 - generally runs faster than interpreted code
 - examples: C, C++, Java, Go

Intermezzo: Popularity of languages

- As of recently, Python is the most popular programming language...

Mar 2026	Mar 2025	Change	Programming Language		Ratings	Change
1	1			Python	21.25%	-2.59%
2	4	^		C	11.55%	+2.02%
3	2	v		C++	8.18%	-2.90%
4	3	v		Java	7.99%	-2.37%
5	5			C#	6.36%	+1.49%
6	6			JavaScript	3.45%	-0.01%
7	9	^		Visual Basic	2.50%	-0.02%
8	8			SQL	2.00%	-0.57%
9	16	^^		R	1.88%	+0.94%
10	10			Delphi/Object Pascal	1.80%	-0.36%

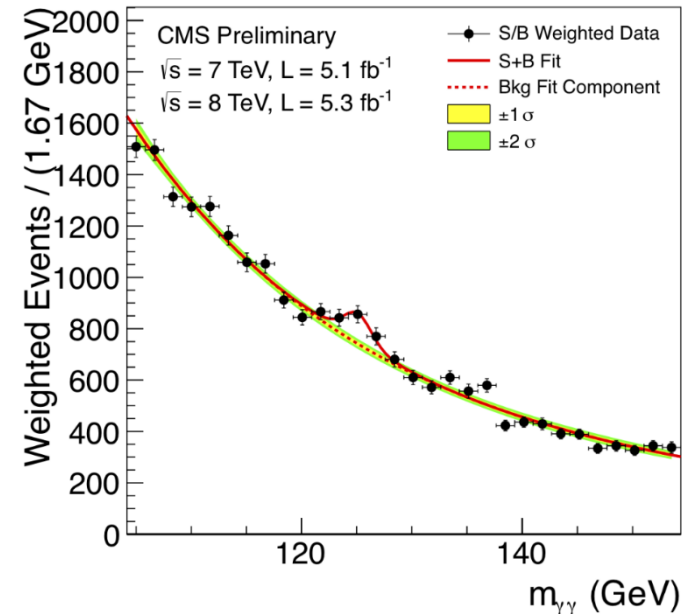
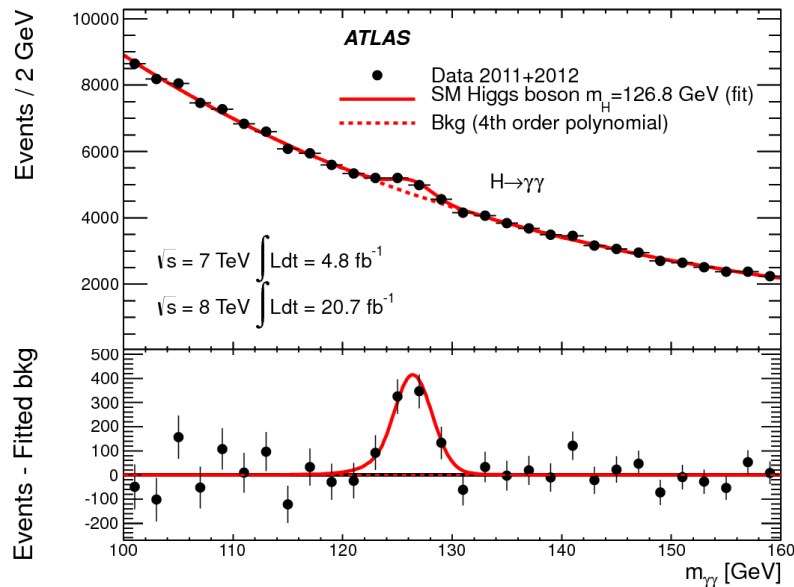
Intermezzo: Popularity of languages

... but some very old languages, like Fortran or COBOL, are still alive!

11	24	⬆️		Perl	1.75%	+1.05%
12	12			Scratch	1.63%	-0.03%
13	11	⬇️		Fortran	1.45%	-0.25%
14	14			Rust	1.31%	+0.09%
15	15			MATLAB	1.29%	+0.31%
16	7	⬇️		Go	1.29%	-1.49%
17	17			Assembly language	1.29%	+0.42%
18	13	⬇️		PHP	1.23%	-0.25%
19	18	⬇️		Ada	1.10%	+0.25%
20	26	⬆️		Swift	1.04%	+0.44%

Free advert (#3)

- Is this a right course for you? ROOT, what's that?
 - If you have ever asked yourself what is the software in which the Higgs discovery plots were actually made ...

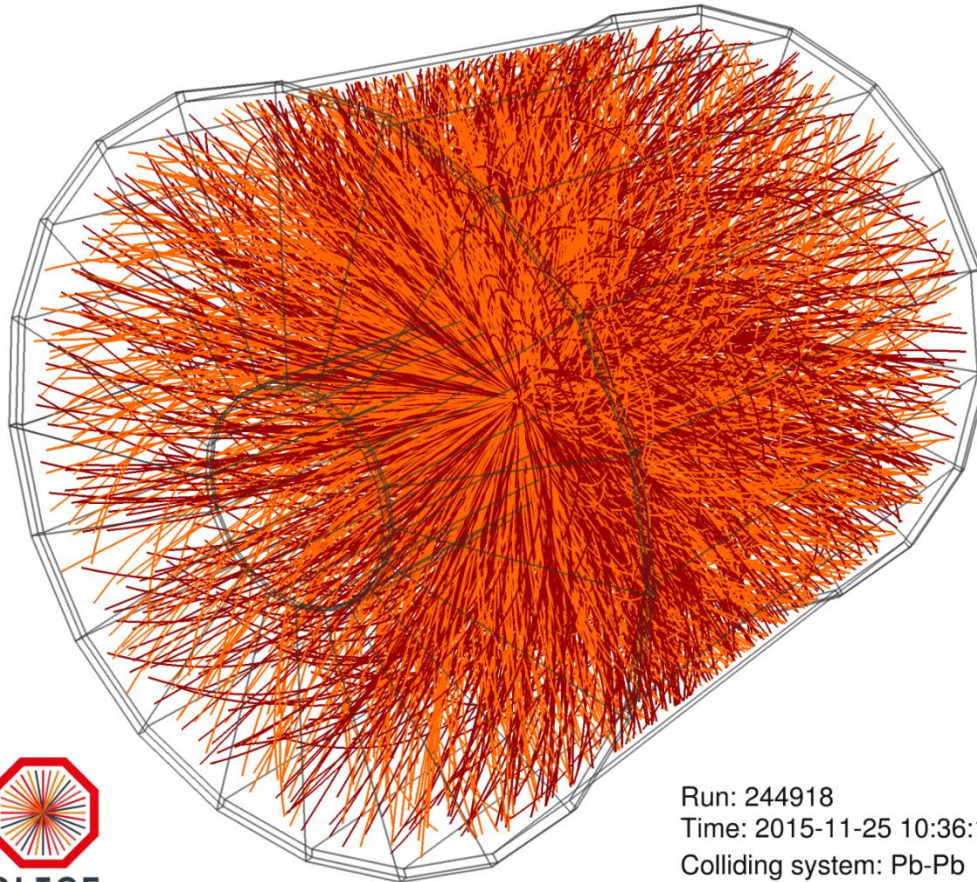


... then this is the right course for you!

- At the very basic level, we can use ROOT for plotting, histogramming, fitting, etc.

Free advert (#3)

- This is the typical heavy-ion event at Large Hadron Collider reconstructed by ALICE Collaboration



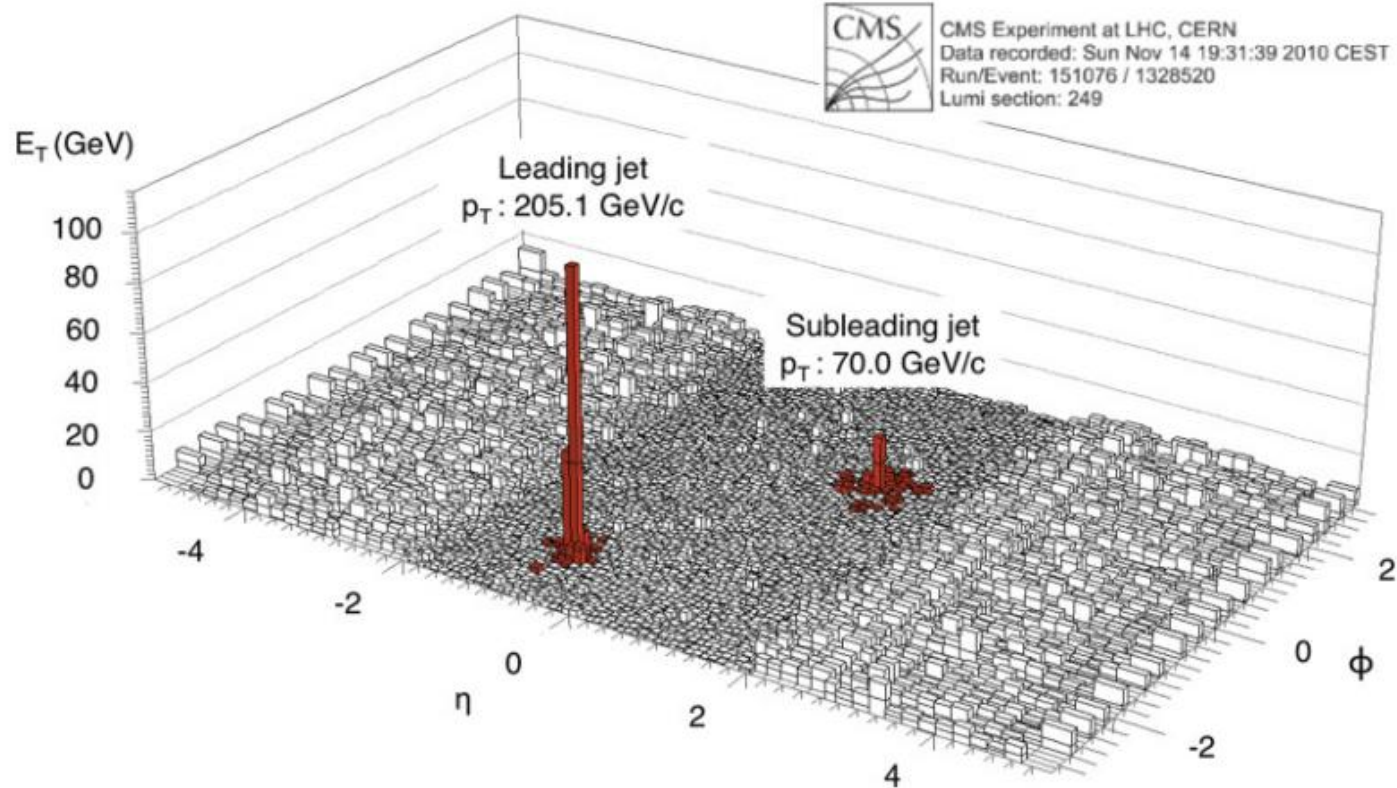
Run: 244918
Time: 2015-11-25 10:36:18
Colliding system: Pb-Pb
Collision energy: 5.02 TeV

Free advert (#3)

- Trajectories of more than 10000 particles are reconstructed by AliROOT (C++ code specific to ALICE Collaboration built on top of ROOT)
- Most important major collaborations worldwide in high-energy physics currently use ROOT
 - Also the future ones (e.g. CBM at GSI is developing CbmRoot)

Free advert (#4)

- In terms of histogramming quality and performance, it's difficult to beat ROOT...



ROOT

- Object-oriented framework, written in C++ and developed at CERN, for data analysis in high-energy physics
- The development was initiated by René Brun and Fons Rademakers in 1994, and is still under active development
 - Latest release: Version 6.36.10 (March 14th, 2026)
- Webpage: <https://root.cern.ch/>
- Root forum: <https://root-forum.cern.ch/>
- Source code: <https://github.com/root-project/root>



Thanks!